EXP.NO: 09	Memory Management Technique
DATE:	

AIM:

To implement and compare the First Fit and Best Fit memory management schemes to understand how different strategies affect block utilization and fragmentation.

ALGORITHM:

Initialize: Define the number of memory blocks and their initial sizes. Define the number of processes and their requested memory sizes.

First Fit Logic:

Iterate through each process.

Traverse the memory blocks from the beginning.

Allocate the process to the **very first** block that satisfies the size requirement.

Update the block size by subtracting the process size to reflect internal fragmentation.

Best Fit Logic:

Iterate through each process.

Search **all** memory blocks to find the one that is large enough but results in the **smallest** leftover fragment.

Allocate the process to that specific block and update its remaining size.

Output: For both methods, print the process-to-block mapping and the resulting fragmentation, or a failure message if no block fits.

CODE:

FF

```
#include <stdio.h>
int main() {
   int m,n;
   scanf("%d",&m);
   int b[m],p;
```

```
    for(int i=0;i<m;i++) scanf("%d",&b[i]);

    scanf("%d",&n);
    for(int i=0;i<n;i++) {
        scanf("%d",&p);
        int f=0;
        for(int j=0;j<m;j++) {
            if(b[j]>=p) {
                printf("P%d->B%d Frag=%d\n",i+1,j+1,b[j]-p);
                b[j]-=p; f=1; break;
            }
        }
        if(!f) printf("P%d not allocated\n",i+1);
    }
}
```

BF

```
#include <stdio.h>
int main() {
    int m,n;
    scanf("%d",&m);
    int b[m],p;
    for(int i=0;i<m;i++) scanf("%d",&b[i]);

    scanf("%d",&n);
    for(int i=0;i<n;i++) {
        scanf("%d",&p);
        int bi=-1;
        for(int j=0;j<m;j++)
            if(b[j]>=p && (bi==-1 || b[j]<b[bi])) bi=j;

        if(bi!=-1) {
            printf("P%d->B%d Frag=%d\n",i+1,bi+1,b[bi]-p);
            b[bi]-=p;
        } else printf("P%d not allocated\n",i+1);
    }
}
```

OUTPUT:

```
[navdeep@localhost os_exp]$ gcc first_fit.c -o first_fit
[navdeep@localhost os_exp]$ ./first_fit
Enter number of memory blocks:
5
Enter size of each block:
100 500 200 300 600
Enter number of processes:
4
Enter size of each process:
212 417 112 426
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated
[navdeep@localhost os_exp]$ gcc best_fit.c -o best_fit
[navdeep@localhost os_exp]$ ./best_fit
Enter number of memory blocks:
5
Enter size of each block:
100 500 200 300 600
Enter number of processes:
4
Enter size of each process:
212 417 112 426
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with Fragment 174
[navdeep@localhost os_exp]$
```

RESULT:

The program successfully simulates process synchronization. The producer is restricted from adding items when the buffer is full (e=0), and the consumer is restricted from removing items when the buffer is empty (f=0), maintaining data integrity within the shared resource.